JET ENERGY CALIBRATION IN SIMULATION

Mini-project report 30.4.2026

Henry Hiltunen (f0387441) – Lappeenranta–Lahti University of Technology LUT

FY30A1100 From Pulse Shapes to Physics

Abstract

Lappeenranta–Lahti University of Technology LUT

Henry Hiltunen (LUT University)

Jet energy calibration in simulation

Course: FY30A1100 From Pulse Shapes to Physics

13 pages and 3 figures.

Jet energy calibration (JEC) in simulation was done for the mini-project for the course From Pulse Shapes to Physics.

1 Introduction

Article *Jet energy scale and resolution in the CMS experiment in pp collisions at 8 TeV* describes improved corrections in the jet energy scale in CMS experiment's pp collisions at 8 TeV. This mini-project focuses on presenting and rebuilding small part of the jet reconstructions with data gathered during 2024 Khachatryan et al., 2017. The key physics definitions and methods used to study the simulated jet energy response and to derive MC-truth jet energy corrections (JEC) following the CMS article are also described briefly.

Main tasks for the mini-project were to:

1. Measure jet energy response in CMS simulation datasets.
2. From the measurement, derive the MC-truth jet energy corrections.
3. Reproduce the plot shown in the figure 1.1 with new data.

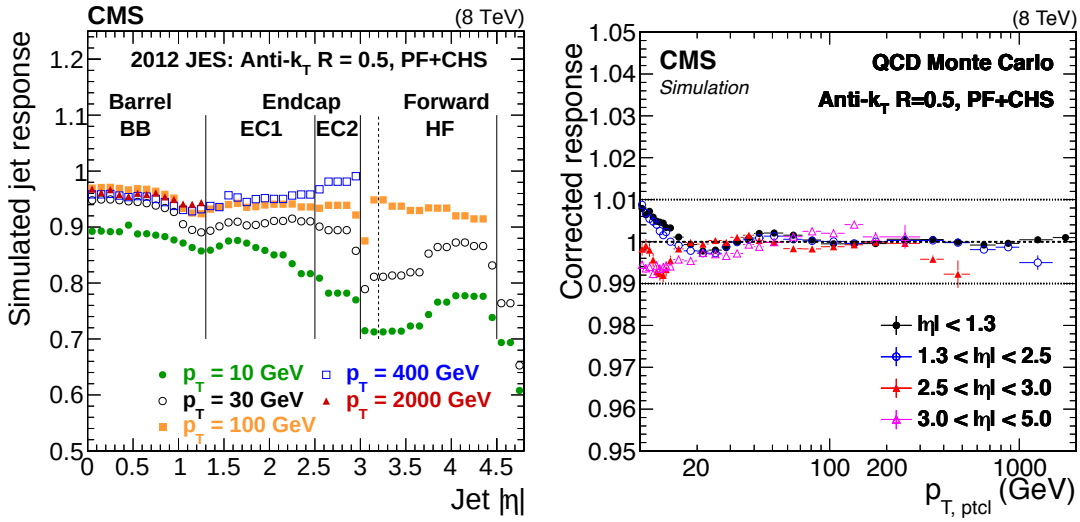

Figure 1.1: Reference figures (Figure 10) at the CMS article: Simulated jet response R_{ptcl} versus $|\eta|$ for $R = 0.5$ (left). Simulated jet response R_{ptcl}, after JEC have been applied, versus $p_{T,ptcl}$ for $R = 0.5$ in various η regions, and with statistical uncertainties (right).

The reproduced figure is not going to be exactly the same due to data being newer than in the article. Also the data size has been reduced to be easily portable between student platforms.

The exact number of events before sample data reduction is not known, but for this mini-project report is was in 28.4 million events (using ROOT command `Events->Print();`) while the article speaks about 10 million events.

1.1 AI usage

It is to be noted, that AI tools have been used to help study the subject and build the scripts.

Copilot 365 (LUT University's Basic-level subscription) was used for initial script generation and Code - OSS's (an open source version of Microsoft's Visual Studio Code) integrated AI models/agents Claude and Copilot (free versions) for adding small tweaks to the code and easing the code development in the development environment.

Several iterations and background checks were done if small details were needed to be added.

In all cases, the article was used as a source to further keeping the knowledge in similar level between AI models.

1.2 Code sharing

As per request, an open Github has been provided, and code shared. Since the data is rather large (over 11 GB) it is not shared, since Github restricts size of the uploaded to the repository Hiltunen, 2026.

2 Theory

The section 5 in the article describes the method to derive the simulated jet response from the given data. The given sample data from quantum chromodynamics (QCD) multijet events from 2024 events.

The Particle-flow (PF) is a reconstruction for the particular detector (CMS) and it combines information from the detector subsystems, such as tracker, calorimeters (ECAL and HCAL), and other systems to identify and measure individual particles. This is generally an improvement for linearity and resolution. For the CMS the PF algorithm took several years to "hone" it down. Anti-k_T jets are simulated with a distance parameter $R = 0.5$.

As mentioned in the CMS article, jets pileup offset corrections were assumed to be corrected in the datafile as well. Otherwise jet area would be needed, which were not present in the datafile, also it was not given as a task.

Event cleaning were not used, though it is possible to switch ON in the scripts if wanted. Next we take a look in the Python scripts.

3 Scripting

The python was used to generate .root files which then can be plotted using the python ROOT library. RDataframes were used to skim through the huge amount of data with proper filtering procedures and ROOT C++ helper scripts inside the python code.

The plot for the article's figure 10 (left) was trying to be matched as well as could. The resulting plot is presented in the figure 3.1. Before it is presented, the code is gone through in detail. Only the most self-explanatory sections are skipped, such as `#include ROOT` and file handling/saving.

3.1 C++ helper functions

The C++ helper functions are used to rapidity-azimuth plane, so it is angular resolution algorithm for jet distance parameter R.

Wrapped in `ROOT.gInterpreter.Declare()`, the C++ language is then used:

```
#include <cmath>
#include "ROOT/RVec.hxx"

using ROOT::VecOps::RVec;

float DeltaPhi(float phi1, float phi2) // Basic angle helpers
{
    float dphi = phi1 - phi2;
    while (dphi >  M_PI) dphi -= 2.f*M_PI;
    while (dphi <= -M_PI) dphi += 2.f*M_PI;
    return dphi;
}

float DeltaR(float eta1, float phi1, float eta2, float phi2) // Basic
    angle helpers
{
    const float deta = eta1 - eta2;
    const float dphi = DeltaPhi(phi1, phi2);
    return std::sqrt(deta*deta + dphi*dphi);
}
```

3.1.1 Mask matching

Masks for the reconstructed jets (shortly referenced as reco) are skimmed through with the RDataframe with C++ helper files as well.

The jets are matched when they will fulfil all the if-else statements:

1. Does this reconstructed jet have a valid matched GenJet index?
2. Is the index inside the GenJet collection?
3. Is the reconstructed jet within $|\eta|$ <5.2?
4. Is the reconstructed/gen pair within ΔR <0.25?

```
RVec<char> BuildMatchedJetMaskFig10(const RVec<short>& jet_gen_idx,
                                    const RVec<float>& jet_eta,
                                    const RVec<float>& jet_phi,
                                    const RVec<float>& genjet_eta,
                                    const RVec<float>& genjet_phi,
                                    int nGenJet)
{
    const int igen = jet_gen_idx[i];
    if (igen < 0) continue;
    if (igen >= nGenJet) continue;
    if (std::abs(jet_eta[i]) >= 5.2f) continue;
    const float dr = DeltaR(jet_eta[i], jet_phi[i],
                            genjet_eta[igen], genjet_phi[igen]);
    if (dr >= 0.25f) continue;
        mask[i] = 1;
    return mask;
}
```

3.1.2 Vector helper functions

`RVec<float> ApplyMaskFloat(const RVec<float>& v, const RVec<char>& mask)` is used for checking that previously built masks are actually useful for us in the plotting phase, meaning if the mask for the reconstructed jet is used or not.

Next `RVec<float> TakeMatchedGenFloat(...)` creates a vector of matched jet p_T values. For selected reconstructed jets, it uses genJetIdx to fetch the corresponding GenJet quantity. These are saved in the RVec formats.

For corrected p_T second plot was built as well. These are compared in the next section.

For actual raw transverse momentum calculation per jet, the raw values p_T^{raw} are calculated from corrected transverse momentum values p_T^{corr}. These are calculated as:

$$p_T^{raw} = p_T^{corr}(1 - rawFactor) \tag{3.1}$$

```
RVec<float> RawPt(const RVec<float>& jet_pt,
                  const RVec<float>& jet_rawFactor)
{
    RVec<float> out;
    out.reserve(jet_pt.size());
    for (size_t i = 0; i < jet_pt.size(); ++i)
    {
        out.push_back(jet_pt[i] * (1.f - jet_rawFactor[i]));
    }
    return out;
}
```

After this the absolute values are taken from reconstructed jet area η.

For helping the plotting and selecting the binning, we select the window mask for the transverse momentums with PtWindowMask. The given binned window for each p_T are in the `PT_WINDOWS_A` constant:

```
// "pT": (low, high)
PT_WINDOWS_A = {
    "10":   (9.0,    11.0),
    "30":   (28.0,   32.0),
    "100":  (90.0,  110.0),
    "400":  (380.0, 420.0),
    "2000": (1900.0, 2100.0),
}

RVec<char> PtWindowMask(const RVec<float>& pt, float low, float high)
{
    RVec<char> mask(pt.size(), 0);
    for (size_t i = 0; i < pt.size(); ++i)
    {
        if (pt[i] >= low && pt[i] < high) mask[i] = 1;
    }
    return mask;
}
```

Finally the sections of the CMS detector are divided in different $|\eta|$ values, such as both endcaps (EC1 and EC2), barrel (BB) and forward (HF).

These regions are given as `ETA_REGIONS_B` constant.

```
ETA_REGIONS_B = {
    "BB":  (0.0, 1.3),
    "EC1": (1.3, 2.5),
```

```
    "EC2": (2.5, 3.0),
    "HF":  (3.0, 4.5),
}
RVec<char> EtaRegionMask(const RVec<float>& abs_eta, float low, float
    high)
{
    RVec<char> mask(abs_eta.size(), 0);
    for (size_t i = 0; i < abs_eta.size(); ++i)
    {
        if (abs_eta[i] >= low && abs_eta[i] < high) mask[i] = 1;
    }
    return mask;
}
```

3.2 Filtering and applying masks

Python script then uses RDataframe to collect the data from the .root file, applies the masks with use of the helper scripts and passes required data to construct the reconstructed jet per particle-level jet pairs (ratio of arithmetic means in bins of particle-level p_T and reconstructed η) as article gives in the equation form at equation 3.2.

$$R^{reco}_{ptcl}([p_{T,ptcl}, \eta]) = \frac{\langle p_T^{reco} \rangle}{\langle p_{T,ptcl} \rangle} = \frac{\sum p_T^{reco}/N}{\sum p_{T,ptcl}/N} = \frac{\sum p_T^{reco}}{\sum p_{T,ptcl}} \tag{3.2}$$

Which seem to be what article tries to convey.

These are then put in the histograms and saved to .root file, which is then used to plot out in different script. This was done because of the making the plot nicer looking without needing to rerun all the filtering and going over the data because of small change in the code.

3.3 Plotting

The plotting script reads the written .root file and plots using ROOT Python library. Since the plot script is larger due to fancy plotting requirements (by the author himself), it was deemed too long to discuss here. It can be seen in the Github repository Hiltunen, 2026.

3.4 Resulting figures

To get rid of large uncertainties in transverse momentum at different parts of the detector as at higher transverse momentums, there are not enough gathered events near the endcaps EC1 and EC2. For this reason, similar jet area $|\eta_{reco}|$ regions of interest (ROI) were used than in

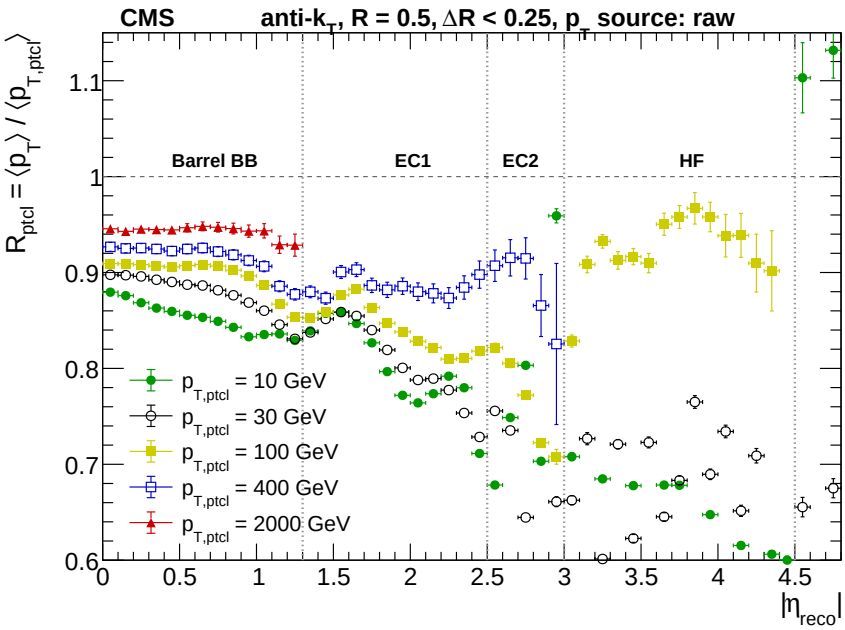

Figure 3.1: Simulated jet response using 2024 data.

the CMS article's plot.

Though we can see that the uncertainties start to cumulate near the end of the given ROIs. Still the plot is close match with articles data.

3.5 Corrected plot

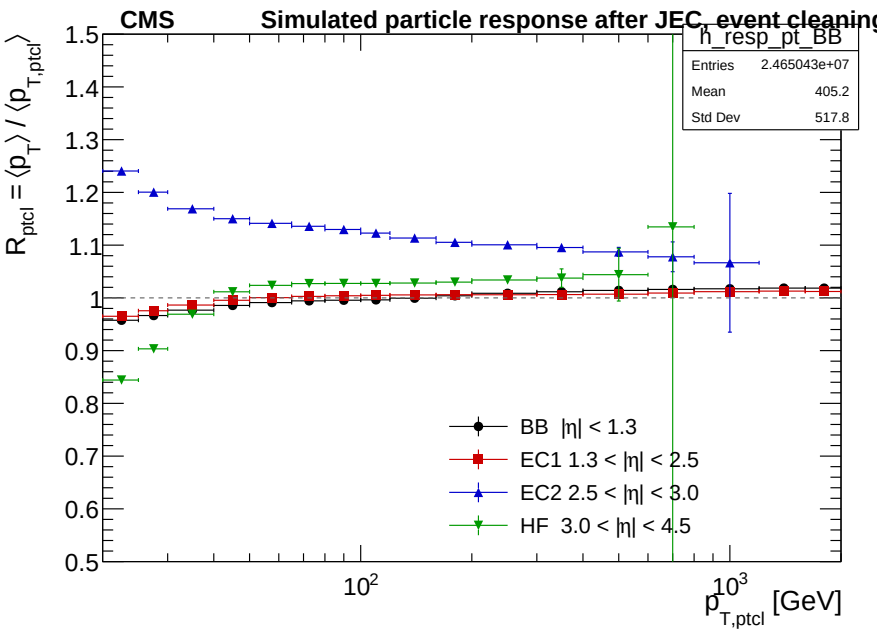

Figure 3.2: Simulated jet response R_{ptcl}, after JEC have been applied using 2024 data.

The code for the figure 10 (right) after drawing the left side was minimal, thus it was also drawn. This is presented in figure 3.2.

For this the p_T were binned from 20 GeV to 2 TeV as article mentions, the bins were:

```
# New binning with more points in low pT:
PT_BINS_B = array("f", [
20, 25, 30, 40, 50, 65, 80, 100, 120,
160, 200, 300, 400, 600, 800, 1200, 1600, 2000])
```

The figure presented is not fully comparable due to binning differences. It would be possible to add more bins to make these similar, but since this was already extra, it was deemed unnecessary.

4 Discussion and conclusions

Comparable results were provided. Differences with newer 2024 data and older 2012 data are are due to pileup, detector used energy range being higher (8 TeV compared to newer 13.6 TeV. Also some of the differences comes due to fact that some of the equipment in the CMS has been changed.

References

Hiltunen, H. (2026). *From-Pulse-Shapes-to-Physics-Course*. [Website]. [Cited: 2026-05-01]. URL: https://github.com/HCharacter/From-Pulse-Shapes-to-Physics-Course.

Khachatryan, V. et al. (Feb. 2017). Jet energy scale and resolution in the CMS experiment in pp collisions at 8 TeV. *Journal of Instrumentation* 12(02), P02014–P02014. ISSN: 1748-0221. DOI: 10.1088/1748-0221/12/02/p02014. URL: http://dx.doi.org/10.1088/1748-0221/12/02/P02014.